SHACL

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

SHACL

https://github.com/mikel-egana-aranguren/ABD

CC BY

SHACL

SHApes Constraint Language

Define la forma que un grafo RDF debería tener: que relaciones, a que nodos, con que valores, etc.

Se validan datos RDF contra esa forma, y se genera un informe que dice si los datos cumplen la forma o no (ValidationReport, en RDF)

SHACL

SHACL es un recomendación (Estándar) del W3C: (W3C)

El vocabulario SHACL, definido en RDF, contiene todo el lenguaje SHACL

El mismo lenguaje RDF para definir datos (RDF) esquema (SHACL) e informes de resultados (ValidationReport) (NoSQL!RDF!)

SHACL vs OWL

Confusión muy común: OWL como lenguaje de esquema

OWL es un lenguaje de inferencia, no de esquema (*): por ejemplo, no dice cuántas relaciones tiene que tener una instancia, si no que si nos encontramos una instancia con esas relaciones, se infiere que pertenece a cierta clase

(*) Se puede conseguir "cerrar" el mundo en OWL, pero con ciertos axiomas y no por defecto

SHACL vs OWL

Confusión muy común: OWL como lenguaje de esquema

SHACL sí es un lenguaje de esquema: por ejemplo, nos dice cuántas relaciones tiene que tener una instancia, y si esa instancia no las tiene, es inválida

Sin embargo, SHACL está evolucionando hacia sustituir a OWL en ciertos contextos

Shape SHACL

```
ex:PersonShape
        a sh:NodeShape ;
        sh:targetClass ex:Person ;
        sh:property [
                sh:path ex:ssn ;
                sh:maxCount 1 ;
                sh:datatype xsd:string ;
                sh:pattern "^\\d{3}-\\d{2}-\\d{4}$" ;
        ] ;
        sh:property [
                sh:path ex:worksFor ;
                sh:class ex:Company ;
                sh:nodeKind sh:IRI ;
        ] ;
        sh:closed true ;
        sh:ignoredProperties ( rdf:type ) .
```

- Nombre del shape (Puede ser cualquiera)
- NodeShape: Captura nodos (Sujeto)
- Este shape se aplica a los nodos de la clase ex:Person (Recursos con rdf:type ex:Person)
- Captura predicados
- Nombre de relación
- Cantidad de relaciones
- Datatype del valor
- Expresión regular para el valor
- rdf:type del objeto del triple
- El nodo tiene que tener IRI (URI), tiene que ser un recurso
- Los nodos que conforman con esta forma no pueden tener más relaciones ...
- ... excepto si son rdf:type

SHACL: Componentes Core

sh:NodeShape: define restricciones sobre nodos del grafo

sh:PropertyShape: define restricciones sobre propiedades

sh:targetClass, sh:targetNode, sh:targetSubjectsOf, sh:targetObjectsOf: qué nodos validar

sh:path: qué propiedad validar

SHACL: Restricciones de cardinalidad

sh:minCount: número mínimo de valores

sh:maxCount: número máximo de valores

```
ex:PersonShape a sh:NodeShape ;
    sh:targetClass ex:Person ;
    sh:property [
        sh:path ex:name ;
        sh:minCount 1 ;
        sh:maxCount 1
    ] .
```

SHACL: Restricciones de tipo

sh:class: el valor debe ser instancia de una clase

sh:datatype: el valor debe tener un datatype específico

sh:nodeKind: tipo de nodo (IRI, Literal, BlankNode)

```
sh:property [
    sh:path ex:birthDate ;
    sh:datatype xsd:date ;
    sh:maxCount 1
] .
```

SHACL: Restricciones de valor

sh:in: el valor debe estar en una lista específica

sh:hasValue: debe existir un valor específico

sh:minExclusive, sh:maxExclusive: rangos numéricos

sh:minInclusive, sh:maxInclusive: rangos numéricos inclusivos

SHACL: Restricciones de cadenas

sh:minLength, sh:maxLength: longitud de strings

sh:pattern: expresión regular que debe cumplir

sh:languageIn: idiomas permitidos para literales

sh:uniqueLang: cada idioma solo puede aparecer una vez

SHACL: Restricciones de cadenas

```
sh:property [
    sh:path rdfs:label ;
    sh:uniqueLang true
] .
```

SHACL: Restricciones lógicas

sh:not: negación de una shape

sh:and: conjunción (todas las shapes deben cumplirse)

sh:or: disyunción (al menos una shape debe cumplirse)

sh:xone: disyunción exclusiva (exactamente una debe cumplirse)

SHACL: Restricciones de forma

sh:node: el valor debe cumplir otra shape

sh:property: define restricciones anidadas

sh:qualifiedValueShape + sh:qualifiedMinCount/MaxCount: cuántos valores deben cumplir una shape

Permite validaciones complejas y composición de shapes

SHACL: Closed Shapes

sh:closed true: solo se permiten las propiedades declaradas

sh:ignoredProperties: lista de propiedades que se ignoran

```
ex:PersonShape a sh:NodeShape ;
    sh:targetClass ex:Person ;
    sh:closed true ;
    sh:ignoredProperties ( rdf:type ) ;
    sh:property [
        sh:path ex:name
    ] .
```

SHACL: Severity Levels

sh:severity: nivel de gravedad de una violación

sh:Violation (por defecto): error crítico

sh:Warning: advertencia

sh:Info: información

Permite diferenciar validaciones obligatorias de recomendaciones

SHACL: Mensajes personalizados

sh:message: mensaje descriptivo para violaciones

Puede ser multilingüe usando language tags

```
sh:property [
    sh:path ex:age ;
    sh:datatype xsd:integer ;
    sh:minInclusive 0 ;
    sh:message "La edad debe ser un entero no negativo"@es ;
    sh:message "Age must be a non-negative integer"@en
] .
```

SHACL: Property Paths

sh:path soporta caminos complejos:

Secuencia: (ex:parent ex:age) - padre.edad

Alternativa: [sh:alternativePath (ex:email ex:phone)]

SHACL: Property Paths

Inverso: [sh:inversePath ex:knows]

Zero o más: [sh:zeroOrMorePath ex:parent]

Uno o más: [sh:oneOrMorePath ex:subClassOf]

SHACL-SPARQL

Extensión avanzada: validaciones con queries SPARQL

sh:sparql permite restricciones arbitrariamente complejas

Más expresivo que SHACL Core pero menos eficiente

Útil para validaciones que requieren razonamiento complejo

SHACL-SPARQL: Ejemplo

```
ex:UniqueEmailShape a sh:NodeShape ;
    sh:targetClass ex:Person ;
    sh:sparql [
        sh:message "Email must be unique" ;
        sh:select """
            SELECT $this ?email WHERE {
                $this ex:email ?email .
                ?other ex:email ?email .
                FILTER ($this != ?other)
            }
        """
    ] .
```

SHACL: Validation Report

Resultado de validación en RDF (NoSQL!RDF!)

sh:ValidationReport con sh:conforms (true/false)

SHACL: Validation Report

sh:ValidationResult para cada violación:

- sh:focusNode: nodo que viola la restricción
- sh:resultPath: propiedad violada
- sh:value: valor problemático
- sh:resultMessage: descripción del error

SHACL: Herramientas

SHACL Playground: editor online

pySHACL: validador Python

Apache Jena SHACL: Java

GraphDB, Stardog: soporte SHACL integrado

SHACL: Buenas prácticas

Usar sh:message descriptivos en múltiples idiomas

Organizar shapes en módulos reutilizables

Separar validaciones críticas (Violation) de recomendaciones (Warning)

Documentar shapes con rdfs:label y rdfs:comment

Preferir SHACL Core sobre SHACL-SPARQL cuando sea posible

SHACL en GraphDB

1. Cargar Shapes (RDF)
2. Cargar datos
3. Si son válidos, los datos se cargan
4. Si son inválidos, no se cargan y la razón del fallo aparece en el Validation Report

SHACL en GraphDB

Crear repositorio activando opción SHACL

Los shapes residen en un grafo específico:

- Por defecto http://rdf4j.org/schema/rdf4j#SHACLShapeGraph
- Se pueden definir shapes en cualquier grafo y enlazarlo con los datos:

ex:dataGraph sh:shapesGraph ex:GraphConShapes

[Ejecutar ejemplos ex1, rdf_type]

SHACL 1.2

Working Draft del W3C (marzo 2026), una evolución significativa respecto a SHACL 1.0

Nuevo alcance ampliado: "SHACL se puede usar para **validación**, **inferencia**, **modelado de dominios**, **generación de interfaces de usuario**, **generación de código** e **integración de datos**"

SHACL 1.2

- SHACL 1.2 Core: núcleo del lenguaje
- SHACL 1.2 SPARQL Extensions: definir restricciones basadas en SPARQL
- SHACL 1.2 Rules (SRL): inferencia basada en reglas
- SHACL 1.2 UI: generación de interfaces de usuario
- SHACL 1.2 Profiling: crear perfiles que representan a grafos

SHACL 1.2 Core: Novedades principales

Nuevos mecanismos de **target**: sh:targetWhere, sh:shape

Nuevas restricciones: sh:someValue, sh:singleLine, restricciones de listas

sh:ShapeClass: una clase que es al mismo tiempo una shape

Soporte para **RDF 1.2**: sh:reifierShape

Propiedades no-validantes para IA: sh:intent, sh:agentInstruction

Core: sh:ShapeClass

Una clase que también actúa como shape, simplificando la definición

Todas las instancias de la clase se validan automáticamente contra la shape

Equivale a definir sh:NodeShape + sh:targetClass por separado

Core: sh:ShapeClass

```
ex:Person a sh:ShapeClass, rdfs:Class ;
    sh:property [
        sh:path ex:name ;
        sh:minCount 1 ;
        sh:datatype xsd:string
    ] .
```

Core: sh:targetWhere

Nuevo mecanismo de target basado en **condiciones** de la shape

Los nodos del grafo de datos que cumplan las condiciones serán focus nodes

Core: sh:targetWhere

```
ex:HighScoreShape a sh:NodeShape ;
    sh:targetWhere [
            sh:property [
                sh:path ex:score ;
                sh:minInclusive 90
            ]
        ] ;
            sh:property [
            sh:path ex:award ;
            sh:minCount 1
    ] .
```

Core: sh:shape como target implícito

sh:shape: marca una shape como objetivo implícito, validando automáticamente cualquier nodo que se encuentre en el grafo y que sea candidato

A diferencia de sh:targetClass, no requiere que los nodos sean explícitamente instancias de una clase

Core: sh:shape como target implícito

```
ex:PersonShape a sh:NodeShape ;
    sh:shape true ;
    sh:property [
        sh:path ex:name ;
        sh:minCount 1
    ] .
```

Core: Node Expressions

Permiten **computar dinámicamente** valores de propiedades

sh:values: asigna valores derivados a una propiedad

Funciones: sh:concat, sh:limit, sh:offset, sh:orderBy, sh:count, sh:sum…

Core: Node Expressions

```
ex:Person-fullName a sh:PropertyShape ;
    sh:path ex:fullName ;
    sh:values [
        sh:concat (
            [ sh:path ex:firstName ]
            " "
            [ sh:path ex:lastName ]
        )
    ] .
```

Core: Nuevos niveles de severidad

Además de sh:Violation, sh:Warning, sh:Info, SHACL 1.2 añade:

- **sh:Trace**: para depuración durante desarrollo
- **sh:Debug**: información de diagnóstico

Core: Propiedades para Agentes IA

Nuevas propiedades **no-validantes** para integración con agentes inteligentes:

- **sh:intent**: describe la intención de la shape (para humanos y agentes)
- **sh:agentInstruction**: instrucciones específicas para agentes IA/LLM

Core: sh:singleLine y sh:conformanceDisallows

sh:singleLine: indica si un valor literal debe ser una sola línea (sin saltos de línea). Usado por interfaces de usuario para elegir el widget adecuado

sh:conformanceDisallows: permite definir qué impide la conformancia, ampliando el modelo de validación

SHACL 1.2 Rules

Nueva especificación para **inferencia basada en reglas**

Introduce el **Shape Rules Language (SRL)**

Las reglas se pueden adjuntar a shapes para generar nuevos triples

Soporta **recursión** y **negation as failure**

Rules: Sintaxis SRL

Las reglas usan sintaxis **RULE/WHERE**:

La cláusula WHERE define condiciones; la cláusula RULE define los triples a generar

Rules: Sintaxis SRL

```
PREFIX ex: <http://example.com/ns#>

RULE {
    ?person ex:fullName ?fullName .
}
WHERE {
    ?person ex:firstName ?first .
    ?person ex:lastName ?last .
    BIND(CONCAT(?first, " ", ?last) AS ?fullName)
}
```

Rules: Negación y recursión

Negation as failure: reglas para cuando algo NO existe

Recursión: las reglas pueden generar triples que activan otras reglas, con **estratificación** para garantizar resultados consistentes

Rules: Adjuntar reglas a Shapes

Las reglas se pueden vincular a shapes específicas, para que solo se apliquen a los focus nodes de la shape

```
ex:PersonShape a sh:NodeShape ;
    sh:targetClass ex:Person ;
    sh:rule [
        a sh:SRLRule ;
        sh:construct """
            RULE { $this ex:fullName ?fn }
            WHERE {
                $this ex:firstName ?f .
```

Rules: Estratificación

Las reglas se evalúan en **capas de estratificación** para resolver dependencias y negación:

1. Se analiza el grafo de dependencias entre reglas
2. Las reglas que dependen de negaciones de otras se colocan en capas superiores
3. Cada capa se evalúa completamente antes de pasar a la siguiente

Garantiza un modelo semántico único y consistente

SHACL 1.2 SPARQL Extensions

Define extensiones de SHACL basadas en **SPARQL 1.2**

- **Restricciones basadas en SPARQL** (sh:sparql)
- **Componentes de restricción** reutilizables (sh:ConstraintComponent)
- **Expresiones de nodo SPARQL** (sh:SelectExpression, sh:SPARQLExprExpression)

Más expresivo que SHACL Core pero menos eficiente

SPARQL: Restricciones con sh:sparql

Permiten restricciones **arbitrariamente complejas** usando SELECT queries

La variable **$this** se pre-asocia al focus node actual

SPARQL: Restricciones con sh:sparql

```
ex:LanguageShape a sh:NodeShape ;
    sh:targetClass ex:Country ;
    sh:sparql [
        sh:message "Debe tener etiqueta en alemán" ;
        sh:prefixes ex: ;
        sh:select """
            SELECT $this ?value WHERE {
                $this ex:germanLabel ?value .
                FILTER (!langMatches(lang(?value), "de"))
            }
        """
    ] .
```

SPARQL: Componentes de restricción

Permiten **abstraer y reutilizar** lógica de validación SPARQL

Se declaran con sh:ConstraintComponent + sh:parameter + validador

SPARQL: Componentes de restricción

```
ex:LangComponent a sh:ConstraintComponent ;
    sh:parameter [ sh:path ex:lang ] ;
    sh:validator [
        a sh:SPARQLAskValidator ;
        sh:message "Idioma incorrecto: {$lang}" ;
        sh:ask """ASK {
            FILTER (isLiteral($value) &&
              langMatches(lang($value), $lang))
        }"""
    ] .
```

SPARQL: Validadores ASK vs SELECT

Dos tipos de validadores SPARQL:

- **ASK** (sh:SPARQLAskValidator): devuelve true si el valor **cumple** la restricción
- **SELECT** (sh:SPARQLSelectValidator): devuelve los valores que **NO cumplen**

Selección de validador: sh:nodeValidator (para NodeShape), sh:propertyValidator (para PropertyShape), sh:validator (para ambos, solo ASK)

SHACL 1.2 UI (User Interfaces)

Nueva especificación para **generación de interfaces de usuario** a partir de shapes SHACL

Define el concepto de **SHACL Renderer**: software que procesa shapes y datos para generar UI dinámicamente

Nuevo namespace: **shui:** (http://www.w3.org/ns/shacl-ui#)

Dos modos: **vista** (visualización) y **edición** (modificación de datos)

UI: Widgets

- **Editores** (shui:Editor): para modificar datos — TextFieldEditor, DatePickerEditor, BooleanSelectEditor, AutoCompleteEditor, EnumSelectEditor…
- **Visores** (shui:Viewer): para mostrar datos — LabelViewer, LiteralViewer, HTMLViewer, ImageViewer, URIViewer…

Se seleccionan manualmente (shui:editor, shui:viewer) o automáticamente mediante un **sistema de puntuación**

UI: Ejemplo de editores

```
ex:Person-married a sh:PropertyShape ;
    sh:path ex:married ;
    sh:datatype xsd:boolean .
# → BooleanSelectEditor (score 10)

ex:Person-dateOfBirth a sh:PropertyShape ;
    sh:path ex:dateOfBirth ;
    sh:datatype xsd:date .
# → DatePickerEditor (score 10)

ex:Address-region a sh:PropertyShape ;
    sh:path schema:addressRegion ;
```

UI: Property Paths en modo vista y edición

Modo vista: MUST soportar predicate paths e inverse paths; SHOULD soportar paths complejos (secuencia, alternativa, etc.)

Modo edición: MUST soportar predicate e inverse paths; SHOULD soportar alternative paths

```
ex:PersonShape a sh:NodeShape ;
    sh:property [
        sh:path foaf:name ;
        sh:name "Nombre"
    ] ;
```

UI: Property Roles

Anotaciones para indicar el **rol** de una propiedad en la interfaz

shui:propertyRole: indica cómo debe tratarse una propiedad (ej. como etiqueta de visualización)

```
ex:Label a sh:PropertyShape ;
    sh:path skos:prefLabel ;
    shui:propertyRole shui:LabelRole .
```

Con **prioridad cualificada** (sh:order) y soporte para **anotaciones de triples RDF 1.2**:

UI: DetailsEditor y formularios anidados

shui:DetailsEditor: permite editar nodos complejos como formularios anidados

```
ex:Product-weight a sh:PropertyShape ;
    sh:path ex:weight ;
    shui:editor shui:DetailsEditor ;
    sh:maxCount 1 ;
    sh:node ex:ValueWithWeight ;
    sh:nodeKind sh:BlankNode .

ex:ValueWithWeight a sh:NodeShape ;
```

UI: ValueTableViewer

shui:ValueTableViewer: muestra múltiples valores como una **tabla HTML**

Es un Multi Viewer: toma control sobre todos los valores de una propiedad

Las columnas se derivan de la node shape referenciada con sh:node

El orden de columnas se controla con sh:order

```
ex:Concept-narrower a sh:PropertyShape ;
    sh:path [ sh:inversePath skos:broader ] ;
    sh:name "narrower (tabla)" ;
```

Resumen: Cambios clave en SHACL 1.2

- **Core**: sh:ShapeClass, sh:targetWhere, node expressions, restricciones de listas, soporte RDF 1.2, propiedades para agentes IA
- **Rules**: Shape Rules Language (SRL), recursión, negación, estratificación
- **SPARQL**: annotation properties, SelectExpression, SPARQLExprExpression
- **UI**: SHACL Renderer, widgets con puntuación, property roles, formularios anidados